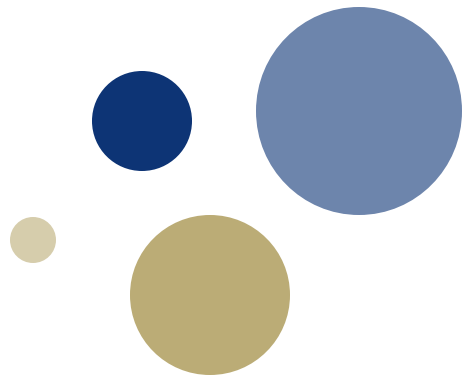




NTNU

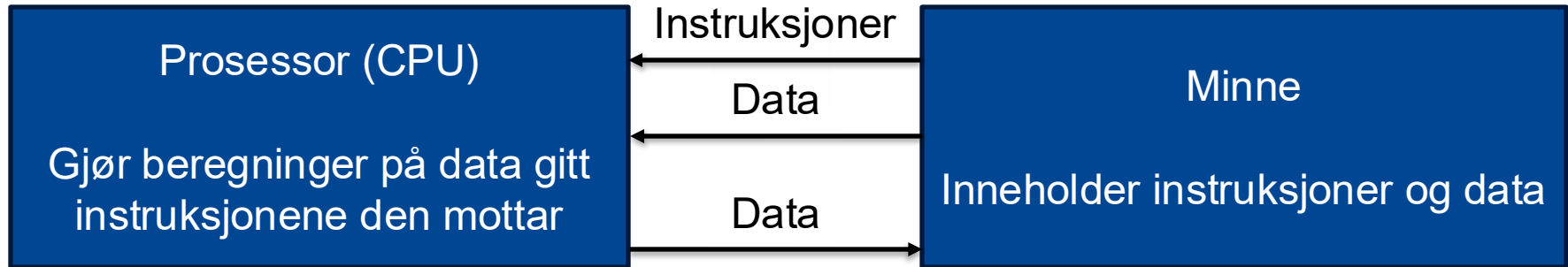
Kunnskap for en bedre verden



Forelesning 7: Samlebåndsprosessorer

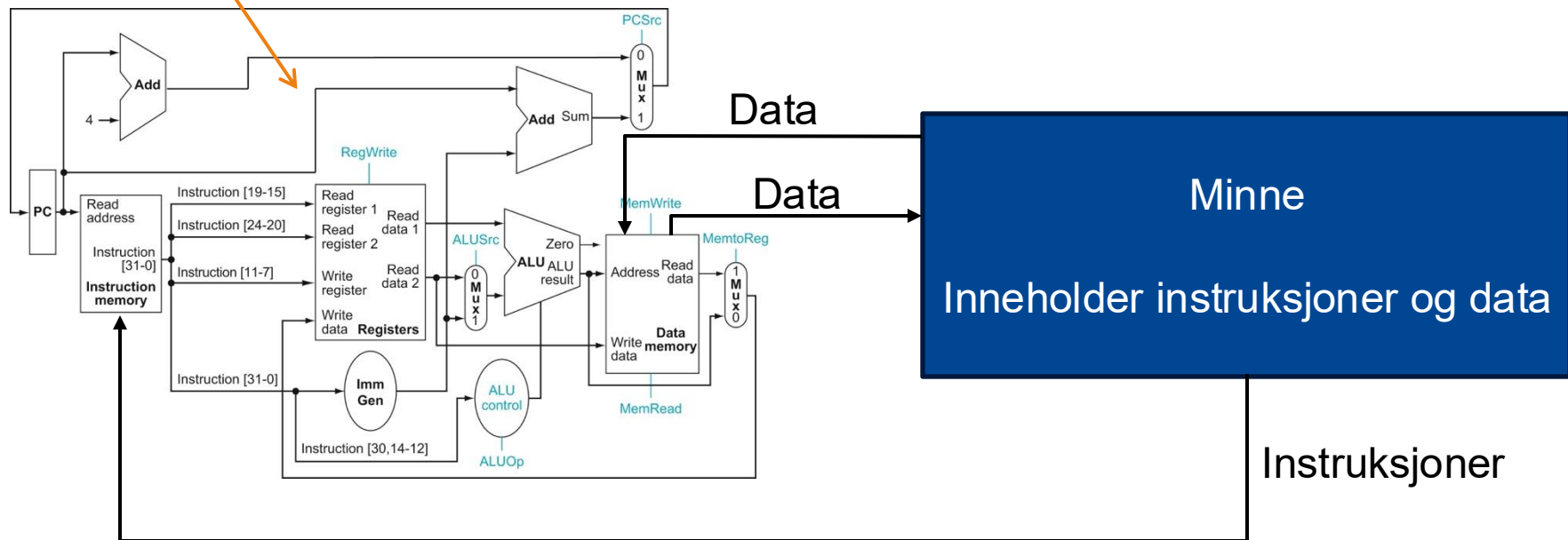
TDT4160 Datamaskiner

Prinsippet om lagrede program



Prinsippet om lagrede program

En enkeltsykelprosessor

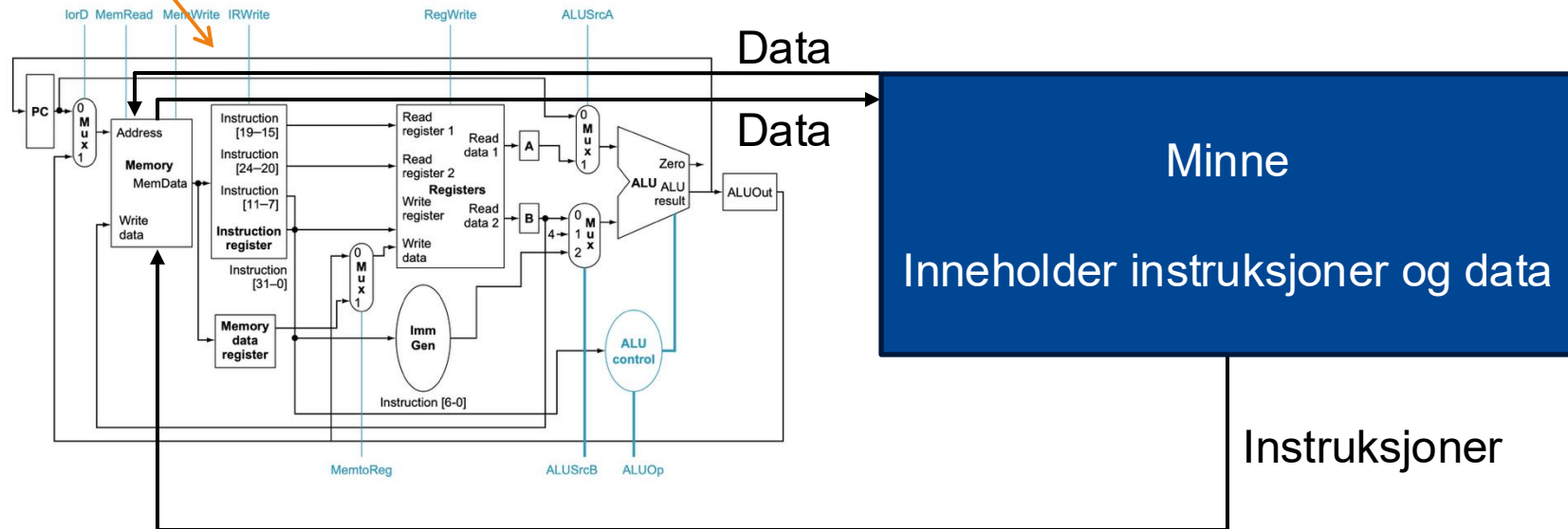


... og vi har begynt jobben med å lage raskere og bedre prosessorer

Prinsippet om lagrede program

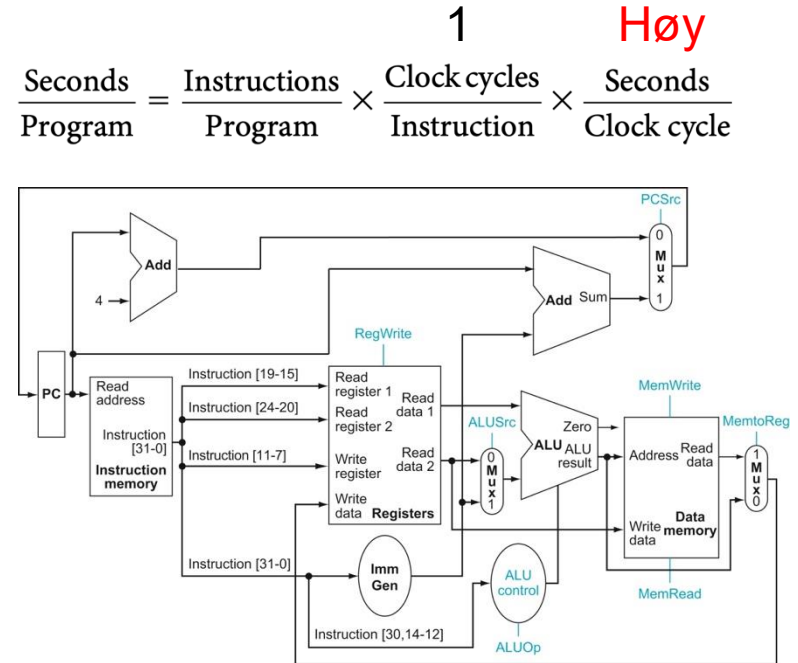
En flersykelprossessor

I dag: Samlebåndsprossessorer



Enkeltskykelprosessoren vår har noen utfordringer...

- Kritisk sti er lang
 - ... og dette gjør at klokkefrekvensen (og dermed ytelsen) blir lav
- Vi må ha flere adderere og minner fordi vi gjør alt i samme klokkesykel
 - Dette har en arealkostnad
 - Hvis vi bruker mer areal, vil vi gjerne også ha høyere ytelse
- Alle instruksjoner tar like lang tid
 - ... og det er ikke akkurat å gjøre det vanlige tilfellet raskt



Flersykelprosessor

- Flersykelprosessoren bruker flere klokkesykler på en instruksjon
 - Klokkefrekvensen kan økes fordi vi gjør mindre arbeid i en klokkesykel
 - Vi kan forenkle datastien fordi vi kan gjenbruke enheter mellom klokkesykler
 - Kontrollenheten blir en tilstandsmaskin

- Dette er den første arkitekturen vi ser på som er i bruk

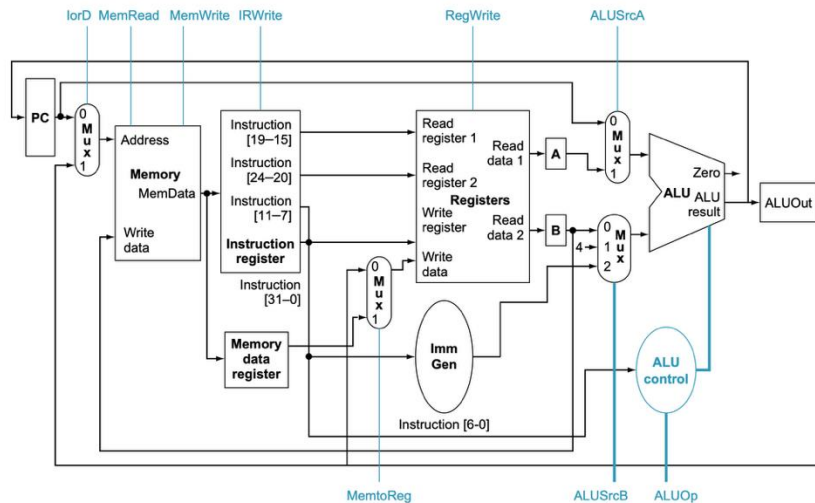
- ... fordi de kan bli (veldig) små
- Små = bruker lite areal på brikken
- Større areal → høyere produksjonskostnad

- Ytelsen er fortsatt ganske dårlig
 - ... og det er ikke lett å gjøre den bedre fordi vi fortsatt kun utfører én instruksjon av gangen

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

Høyere

Lavere





Tema 5.1: Samlebåndsprosessor med 5 steg (Kapittel 4.6)

INTRODUKSJON TIL SAMLEBÅND

Læringsutbytte T5.1

- Studenten skal kunne beskrive **fordeler og ulemper** ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan **oppstartskostnad** og **balanse** mellom steg i samlebåndet påvirker **ytelse**.
- Studenten skal kunne beskrive forskjellen mellom **avhengigheter** og **farer** samt gjengi de tre **hovedgruppene av farer** (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede **strategiene for å håndtere farer** (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende **mikroarkitekturen** til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare **hvordan kontroll implementeres** i 5-stegs samlebåndsprosessoren.
- Studenten skal kunne forklare hvordan **videresending** og **stans** kan brukes til å **håndtere datafarer** samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.
- Studenten skal kunne forklare hvordan **kontrollfarer** kan **håndteres** med **stans** og **prediksjon** samt analysere hvordan strategiene påvirker prosessorens ytelse.

Læringsutbytte T5.1

- Studenten skal kunne beskrive **fordeler og ulemper** ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan **oppstartskostnad** og **balanse** mellom steg i samlebåndet påvirker **ytelse**.
- Studenten skal kunne beskrive forskjellen mellom **avhengigheter** og **farer** samt gjengi de tre **hovedgruppene av farer** (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede **strategiene for å håndtere farer** (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-steps samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare hvordan kontroll implementeres i 5-steps samlebåndsprosessoren.
- Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.
- Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse.

Flersykkelprosessoren vår har dårlig ytelse

- ... og den viktigste grunnen er at den bruker flere klokkesykler på hver instruksjon
- Løsning: Utnytte parallellitet
 - Vi kan jobbe med flere instruksjoner samtidig
 - Instruksjonsnivåparallellitet («Instruction-Level Parallelism (ILP)»)
 - Første skritt: Samlebånd
- De aller fleste prosessorer i bruk i dag er samlebånd्सarkitekturer

Flersykkelprossessor:

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \overset{\text{Høy}}{\frac{\text{Clock cycles}}{\text{Instruction}}} \times \overset{\text{Lav}}{\frac{\text{Seconds}}{\text{Clock cycle}}}$$

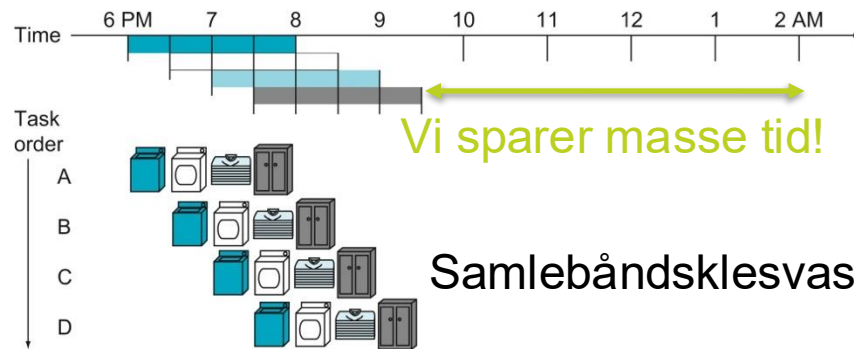
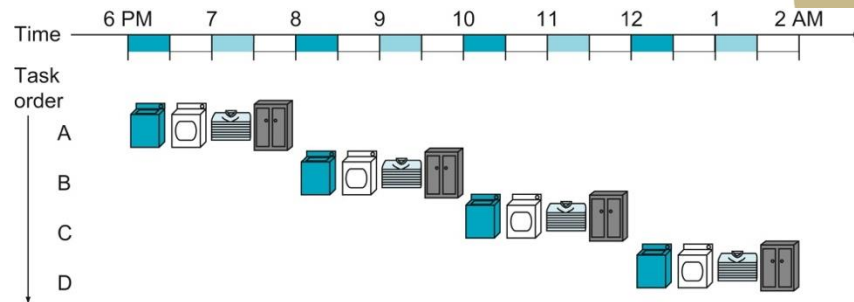
Samlebånd्सprossessor:

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \overset{\text{Lav}}{\frac{\text{Clock cycles}}{\text{Instruction}}} \times \overset{\text{Lav}}{\frac{\text{Seconds}}{\text{Clock cycle}}}$$

Samlebånd: Prinsipp

- Akkurat som i flersykelprosessoren utnytter vi at utføringen av en enkelt instruksjon kan deles opp i ulike steg
 - ... men nå gjør vi skritt 1 for instruksjon B i parallell med for eksempel skritt 2 for instruksjon A
- Analogi: Klesvask
 - Flersykel klesvask:
 - Vi tar ett steg av gangen
 - Vi kan ha en kombinert vaske og tørketrommel
 - Samlebåndsklesvask: Vi kan vaske en haug med klær mens en annen haug med klær er i tørketrommelen

Flersykel klesvask



Samlebåndsklesvask

Herfra blir en vask ferdig i hver «sykel»

Hvorfor samleband?



	CPI	Sykeltid	Areal	Kompleksitet
Enkeltsykelprosessor	Lav	Høy	Lav	Lav
Flersykelprosessor	Høy	Lav	Lavere	Litt høyere
Samlebandsprosessor	Lav til lavere	Lav til lavere	Lav til høy	Middels til høy

Kjøretid



$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

CPI



Sykeltid



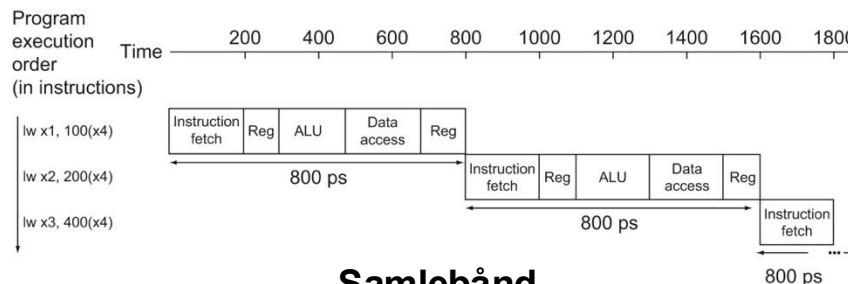
Steg i samlebåndet

1. Instruksjonshenting («Instruction Fetch (IF)»)
2. Instruksjonsdekoding og registerlesing («Instruction Decode (ID)»)
3. Utføring («Execute (EX)»)
4. Minneaksess («Memory access (MEM)»)
5. Tilbakeskriving («Write back (WB)»)

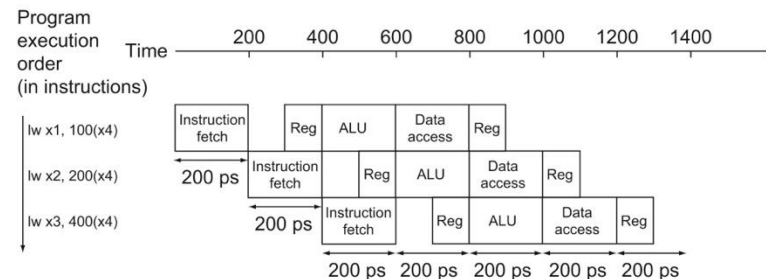
- Disse tilsvarer tilstandene i flersykelprosessen
 - ... men siden alle stegene brukes samtidig må alle instruksjoner innom alle steg
- Dette er en av mange måter å dele opp instruksjonsutføringen på
 - ... og å legge til flere steg i samlebåndet gir både fordeler og ulemper

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Enkeltskykel



Samlebånd



Eksempel: Oppstartskostnad



add x3, x2, x1

add x6, x5, x4

add x8, x7, x6

add x11, x10, x9

add x14, x13, x12

Quiz: Oppstartskostnad

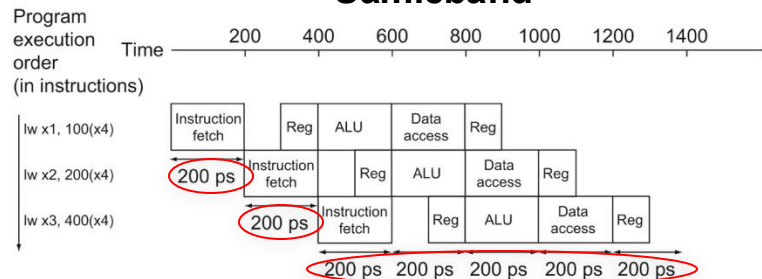


Samlebånd: Ideell ytelse

- Oppstartskostnad
 - Det tar $n-1$ klokkesyklus å fylle et n -stegs samlebånd
- Stegene i samlebåndet må også ta omtrent like lang tid
 - ... fordi hvert steg tar en klokkesykel
 - **Balanse:** Hvor godt lykkes vi med å fordele tiden jevnt mellom stegene i samlebåndet?

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Samlebånd



Perfekt balanse og fullt samlebånd gir:

... og en ideell ytelsesforbedring for et 5-stegs samlebånd på 5X:

$$t_{\text{samlebånd}} = \frac{t_{\text{enkeltskykel}}}{n}$$

$$\frac{800 \text{ ps}}{160 \text{ ps}} = 5 \quad \leftarrow \quad \frac{800 \text{ ps}}{5} = 160 \text{ ps}$$

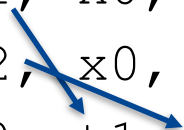
... men vi får «bare» 4X (800/200) fordi samlebåndet vårt ikke er perfekt balansert

Farer («Hazards»)

- Programmet må utføres korrekt også på en samlebåndsprosessor
 - Vi kjører programmet til høyre i Ripes...
- Farer («Hazards») oppstår når et program kan ikke utføres fordi det mangler «noe»
- Klasser av farer:
 - **Strukturfare («Structural hazard»)**: En enhet instruksjonen trenger er ikke tilgjengelig
 - **Datafare («Data hazard»)**: Instruksjonen kan ikke utføres fordi data ikke er tilgjengelig
 - **Kontrollfare («Control hazard»)**: Vi vet ikke hvilken instruksjon som skal utføres
- En avhengighet er en egenskap ved programmet
 - ... mens en fare er en avhengighet som påvirker utføringen av programmet på en gitt maskin

Dataavhengigheter

```
addi t1, x0, 17
addi t2, x0, 13
add s0, t1, t2
```



... som blir datafarer på
5-stegsmaskinen vår

Hvordan håndtere farer?

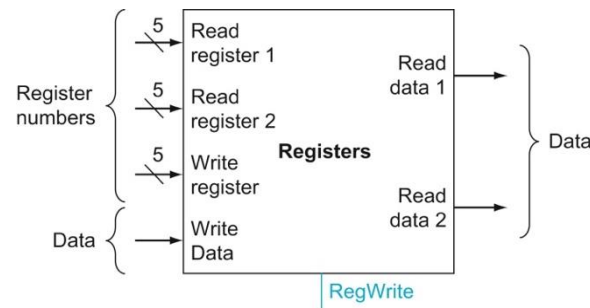
- Unngå ressurskonflikter
- Videresending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)



Hvordan håndtere farer?

- Unngå ressurskonflikter
- Videreending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)

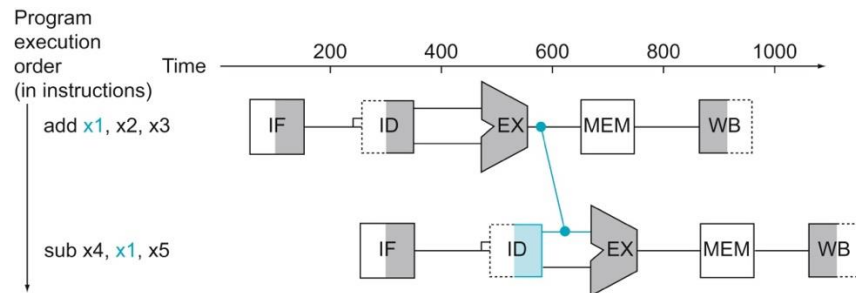
Dette fikser vi med gode arkitekturvalg



Eksempel: Vi lager maskinvaren slik at vi kan lese to registre og skrive ett register i hver klokkesykel

Hvordan håndtere farer?

- Unngå ressurskonflikter
- Videre sending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)

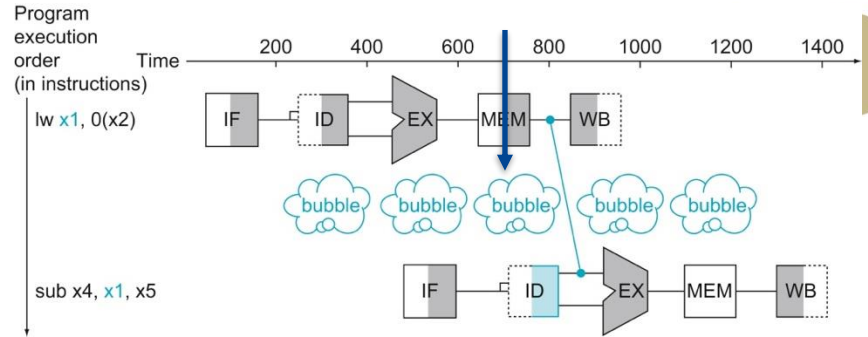


Vi kan legge til ekstra maskinvare som sender data direkte til instruksjonen som trenger dem

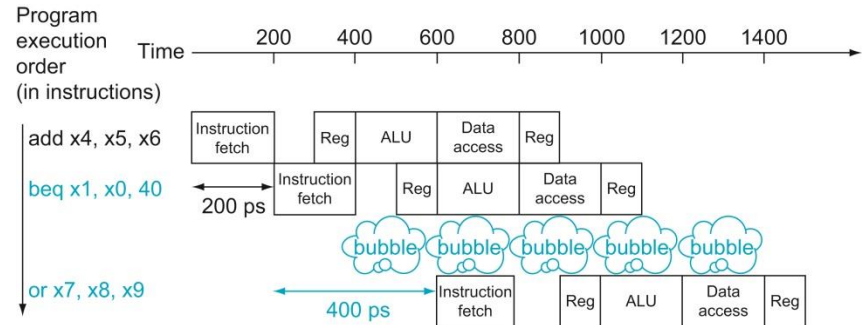
Hvordan håndtere farer?

- Unngå ressurskonflikter
- Videre sending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)

«Hull» i samlebandet



Les-bruk fare («Load-use hazard»)

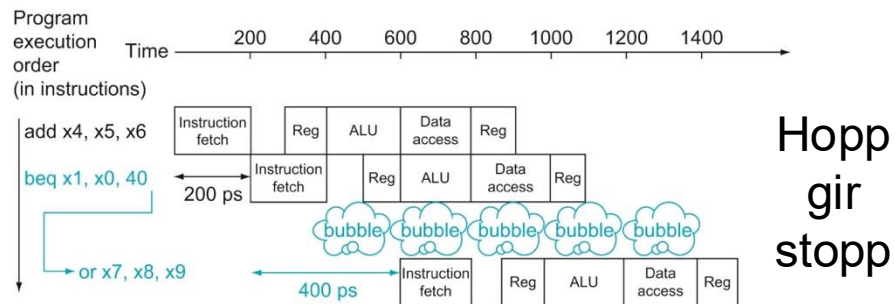
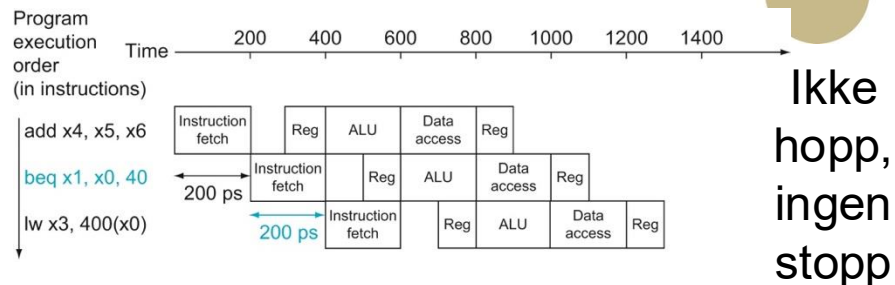


Kontrollfare

Hvordan håndtere farer?

- Unngå ressurskonflikter
- Videresending («Forwarding»)
- Stans («Stalling»)
- Prediksjon («Prediction»)

Prediker ikke hopp



Vi må legge til maskinvare som rydder opp hvis vi gjetter feil



Tema 5.1: Samlebåndsprosessor med 5 steg (Kapittel 4.7)

SAMLEBÅND: DATASTI OG KONTROLL

Læringsutbytte T5.1

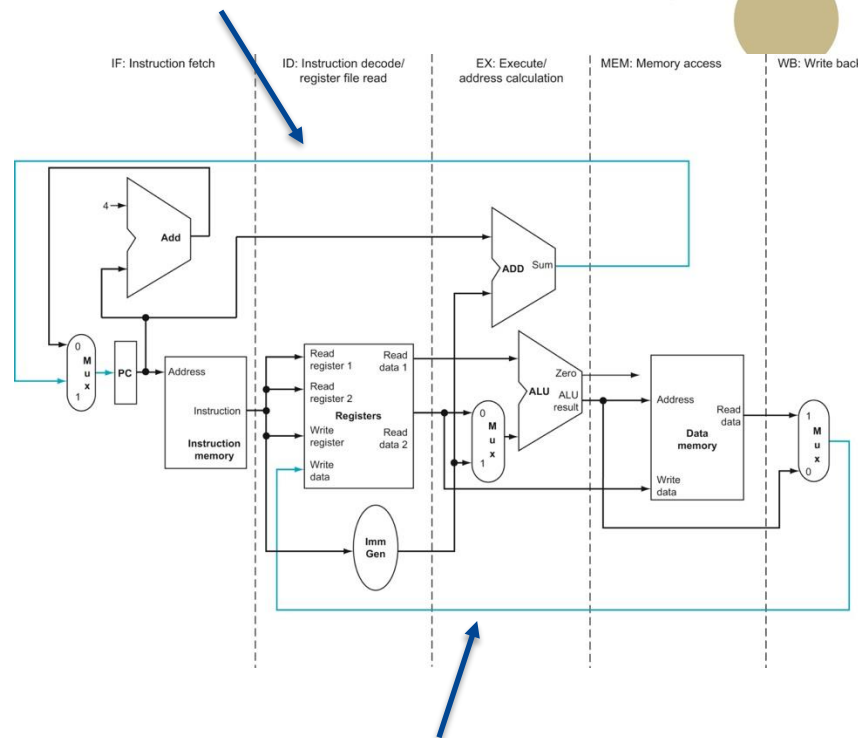
- Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse.
- Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende **mikroarkitekturen** til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare **hvordan kontroll implementeres** i 5-stegs samlebåndsprosessoren.
- Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.
- Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse.

Samlebåndprosessor

Denne tilbakekoblingen lager kontrollfarer

Vi deler opp datastien med registre slik at vi får de 5 skrittene:

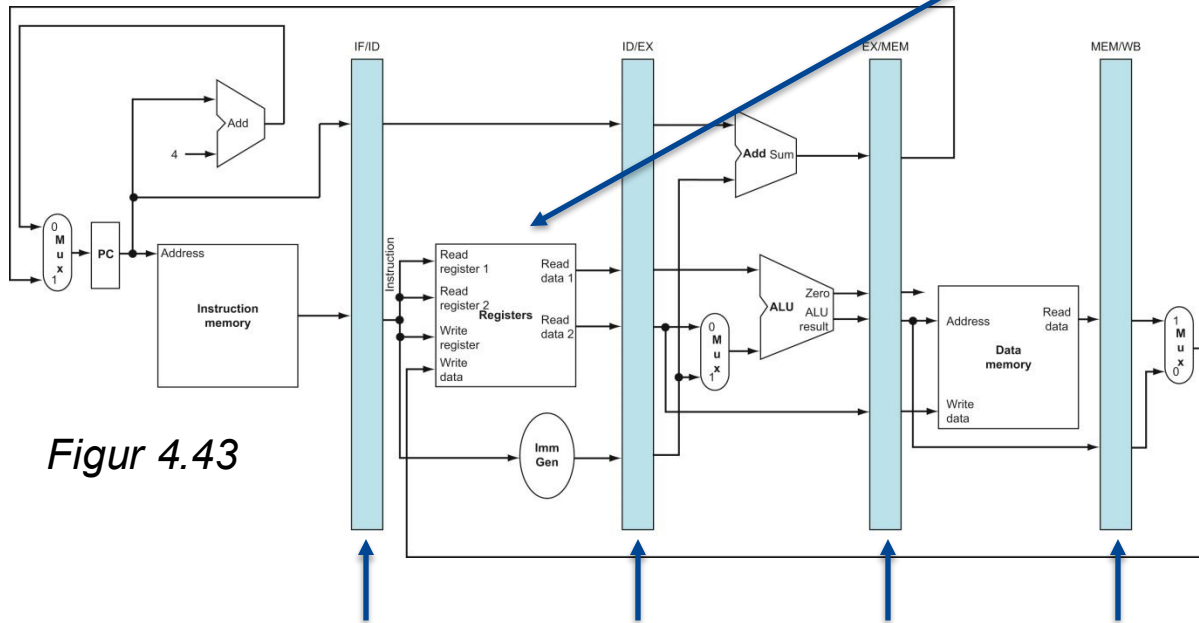
1. Instruksjonshenting («Instruction Fetch (IF)»)
2. Instruksjonsdekoding og registerlesing («Instruction Decode (ID)»)
3. Utføring («Execute (EX)»)
4. Minneaksess («Memory access (MEM)»)
5. Tilbakeskriving («Write back (WB)»)



Denne tilbakekoblingen lager datafarer

Samlebånd: Datasti

Vi skriver i første halvdel
av klokkesyklusen og leser
i andre halvdel



Figur 4.43

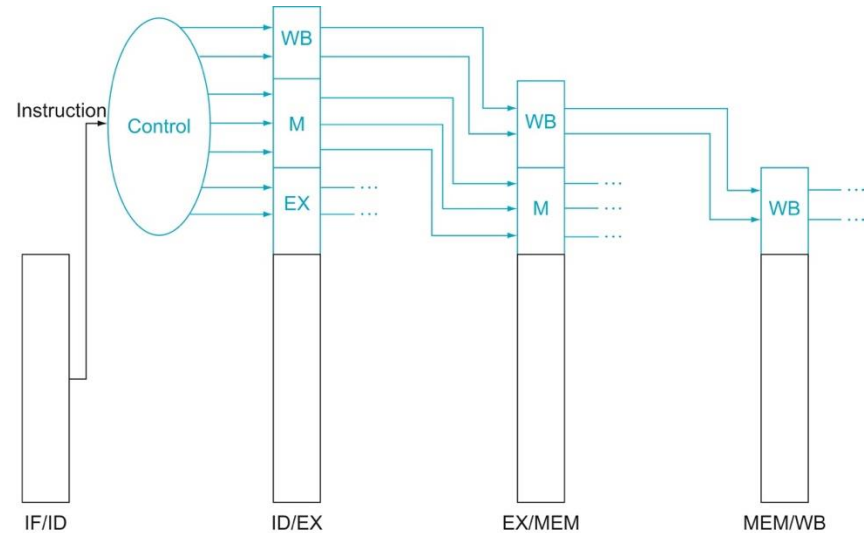
```
addi t1, x0, 17
addi t2, x0, 13
add s0, t1, t2
```

Dette programmet gir feil
svar på grunn av farer
Hvorfor? → Ripes

Samlebåndsregistre lagrer dataverdiene instruksjonen trenger senere

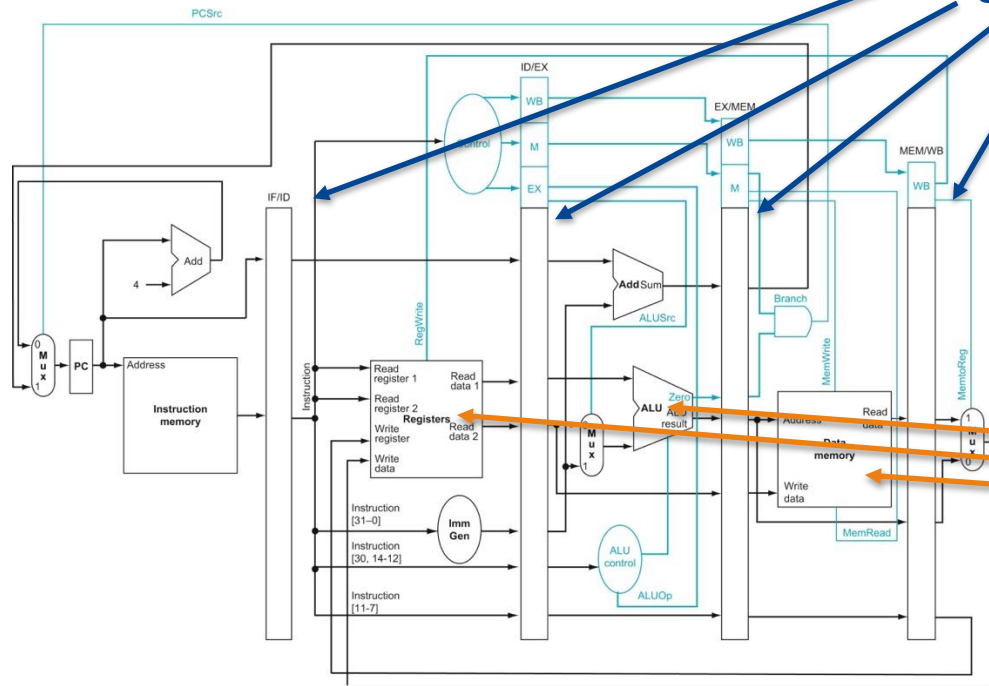
Samlebånd: Kontrollsignaler

- Vi legger kontrollenheten i ID-steget
 - ... og så lar vi kontrollordet følge instruksjonen gjennom samlebåndsregistrene
- Etterhvert som vi kommer lengre ut i samlebåndet har vi brukt flere og flere av kontrollsignalene
 - ... og da trenger vi ikke å huske på dem lengre

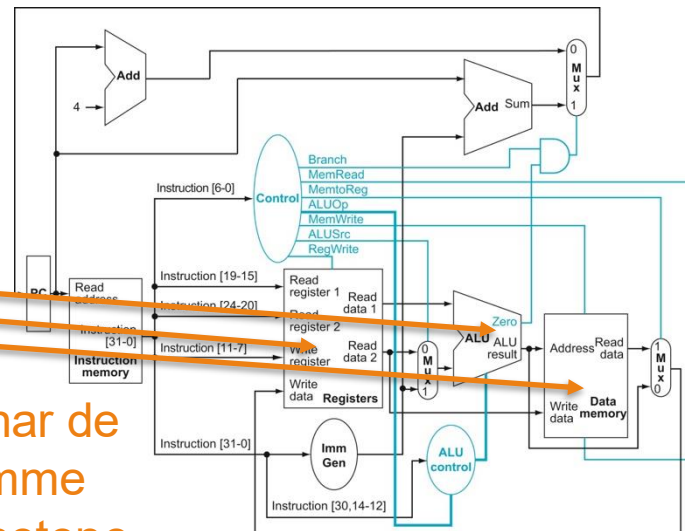


Samlebånd versus enkeltskykel

Forskjellen er
samlebåndsregistrene



5-steps samlebånd



Vi har de
samme
enhetene

Enkeltskykel

Eksempel: Kontrollsignaler

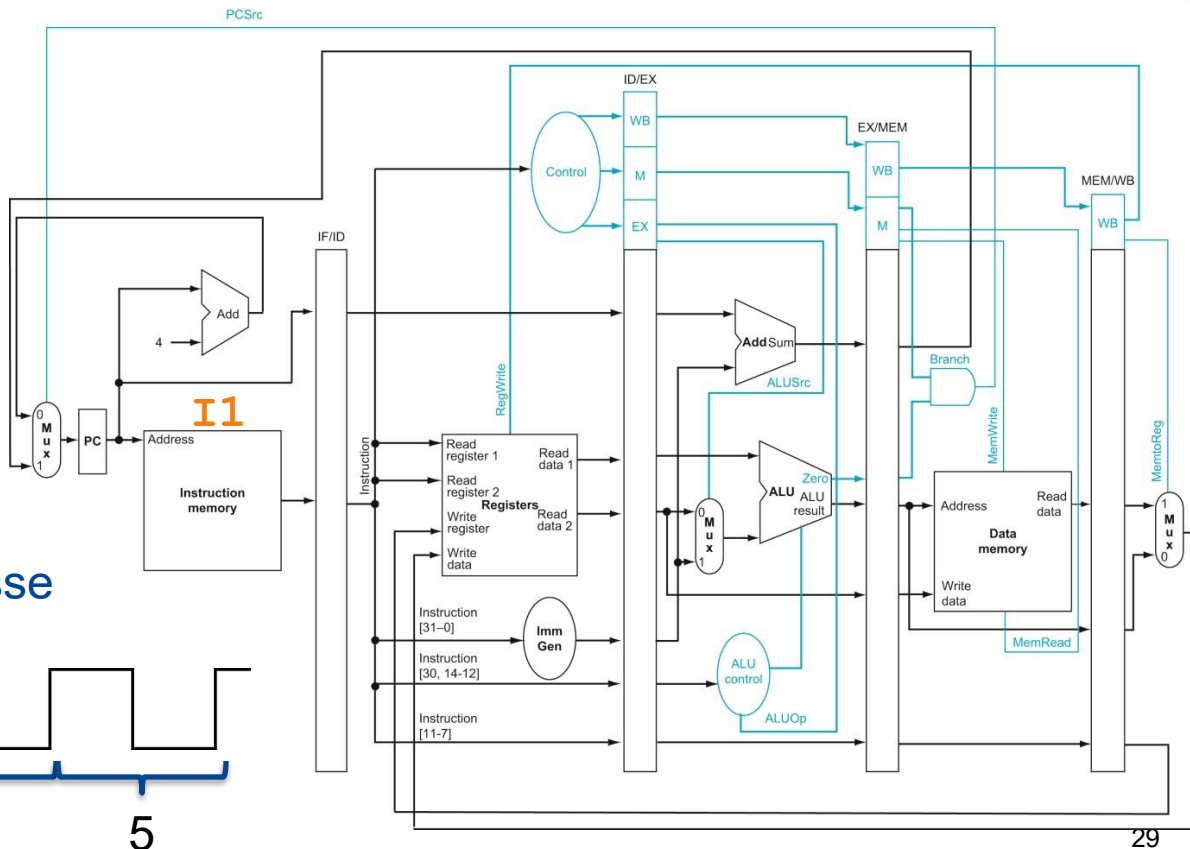
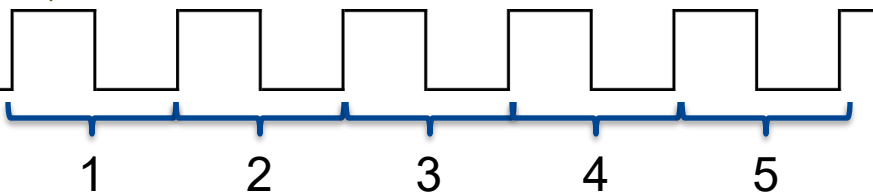
x1	5	x5	
x2	10	x6	-
x3		x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8

Instruksjonsadresse



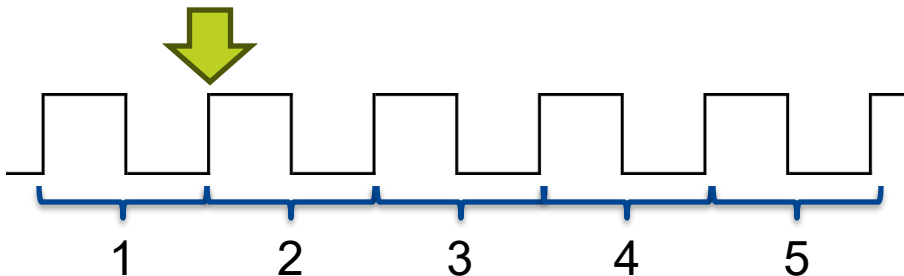
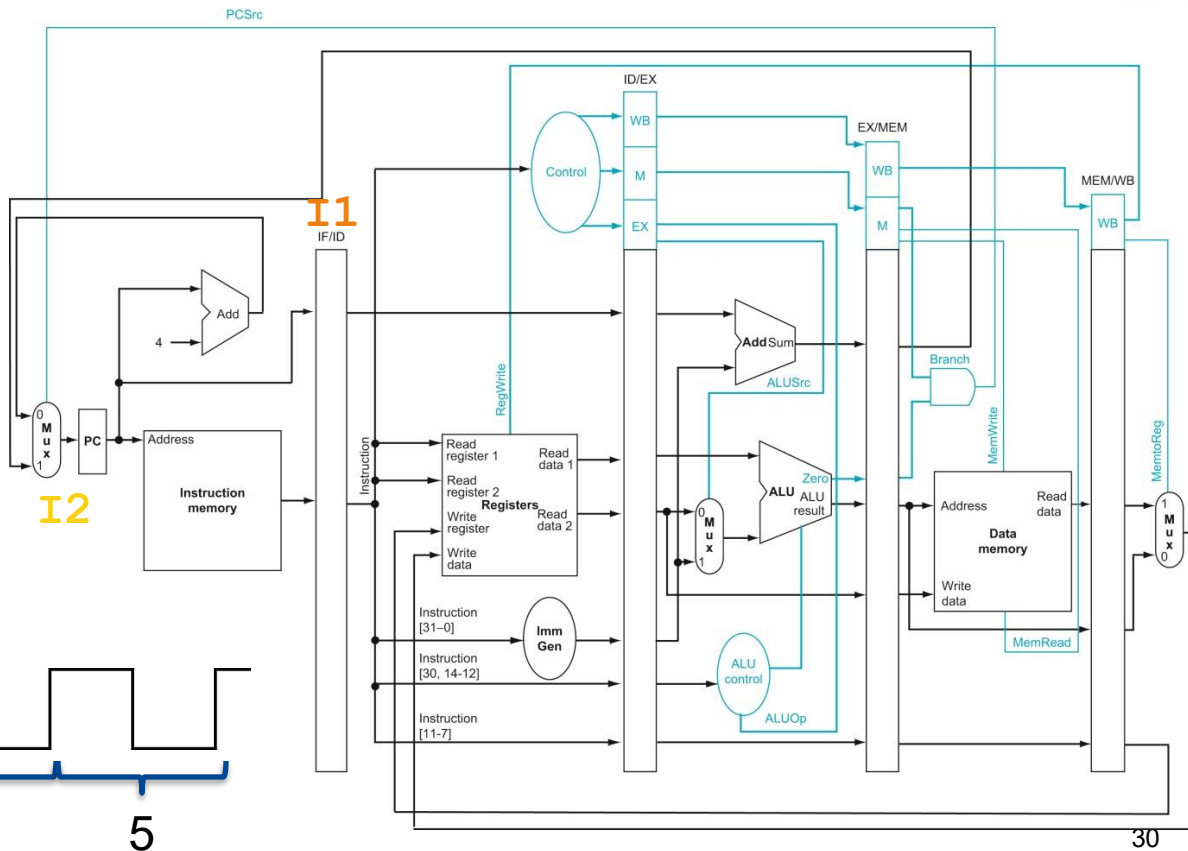
Eksempel: Kontrollsignaler

x1	5	x5	
x2	10	x6	-
x3		x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8



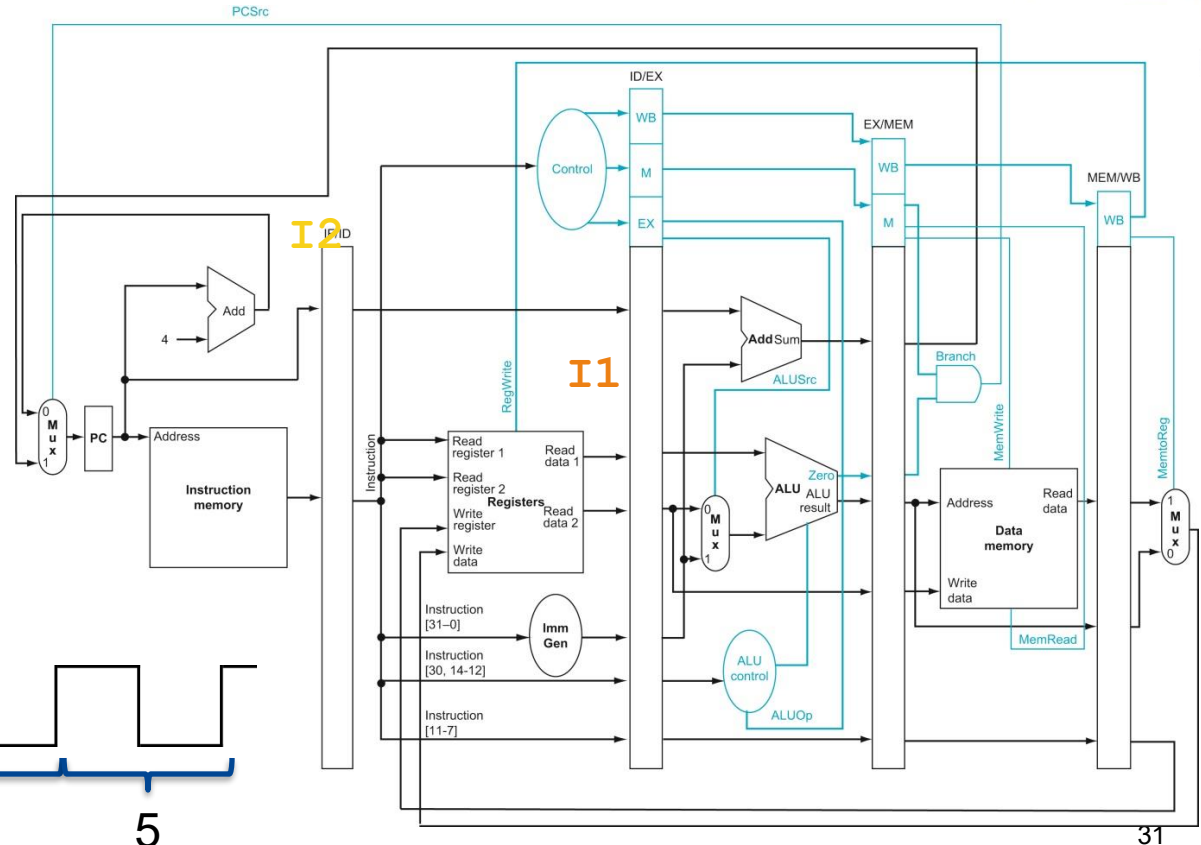
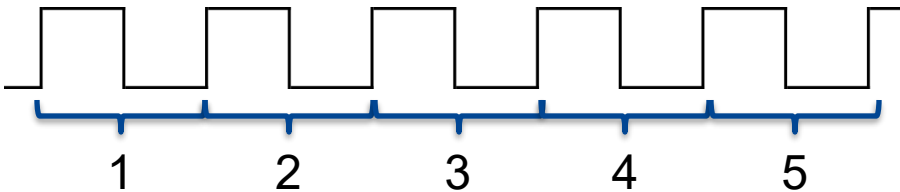
Eksempel: Kontrollsignaler

x1	5	x5	
x2	10	x6	-
x3		x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8



$ALUOp = 00$ for lw

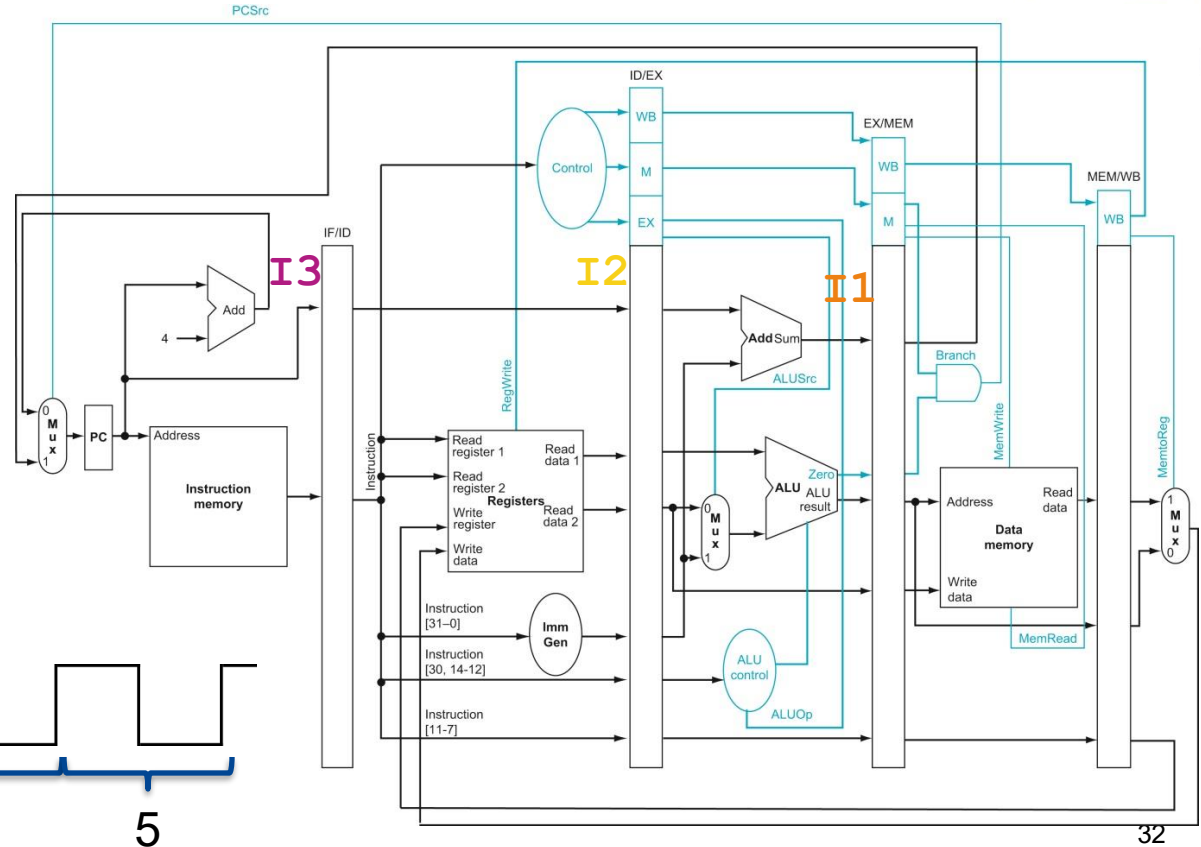
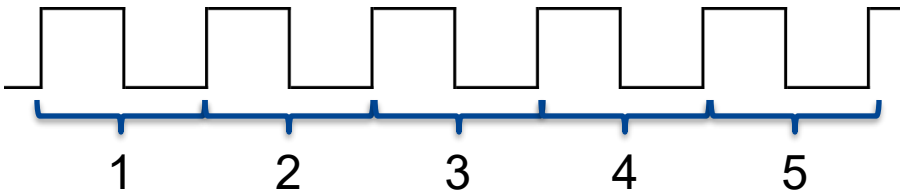
Eksempel: Kontrollsignaler

x1	5	x5	
x2	10	x6	-
x3		x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8



$ALUOp = 01$ for beq

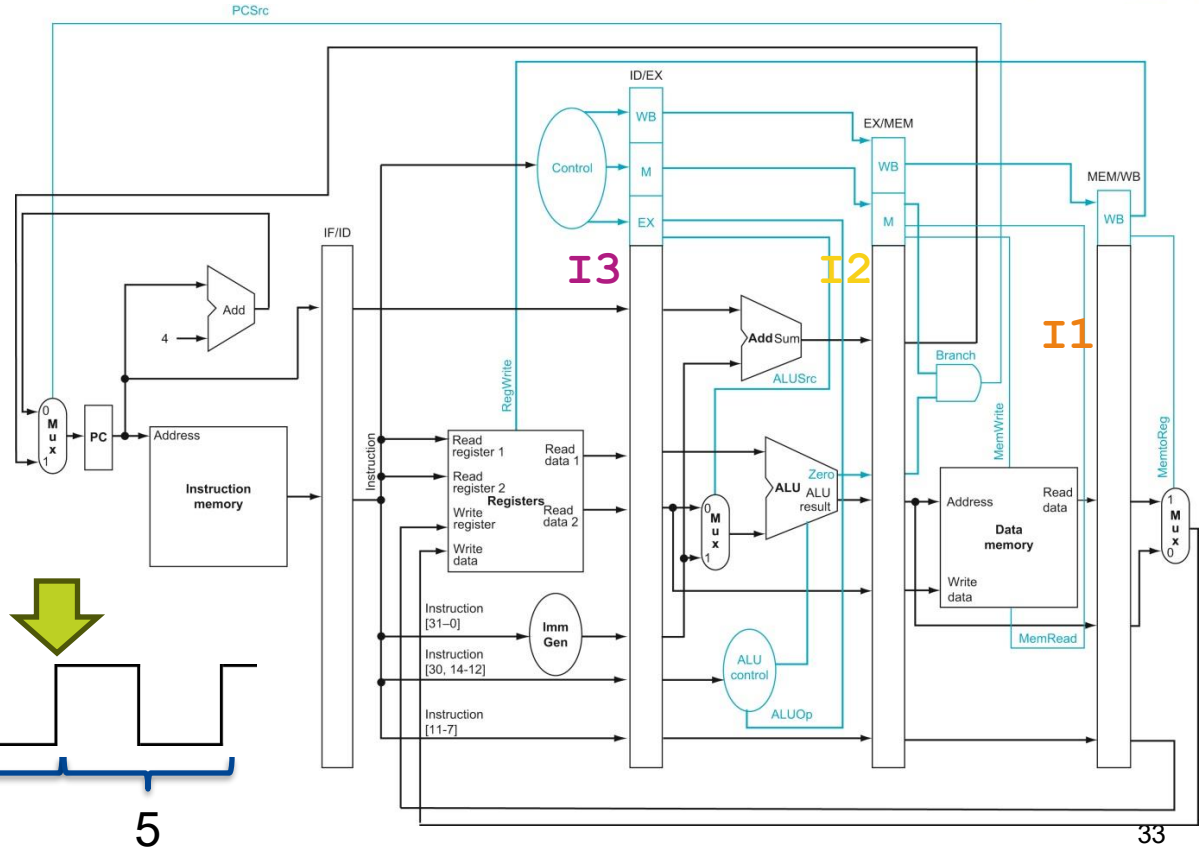
Eksempel: Kontrollsignaler

x1	5	x5	
x2	10	x6	-
x3		x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8



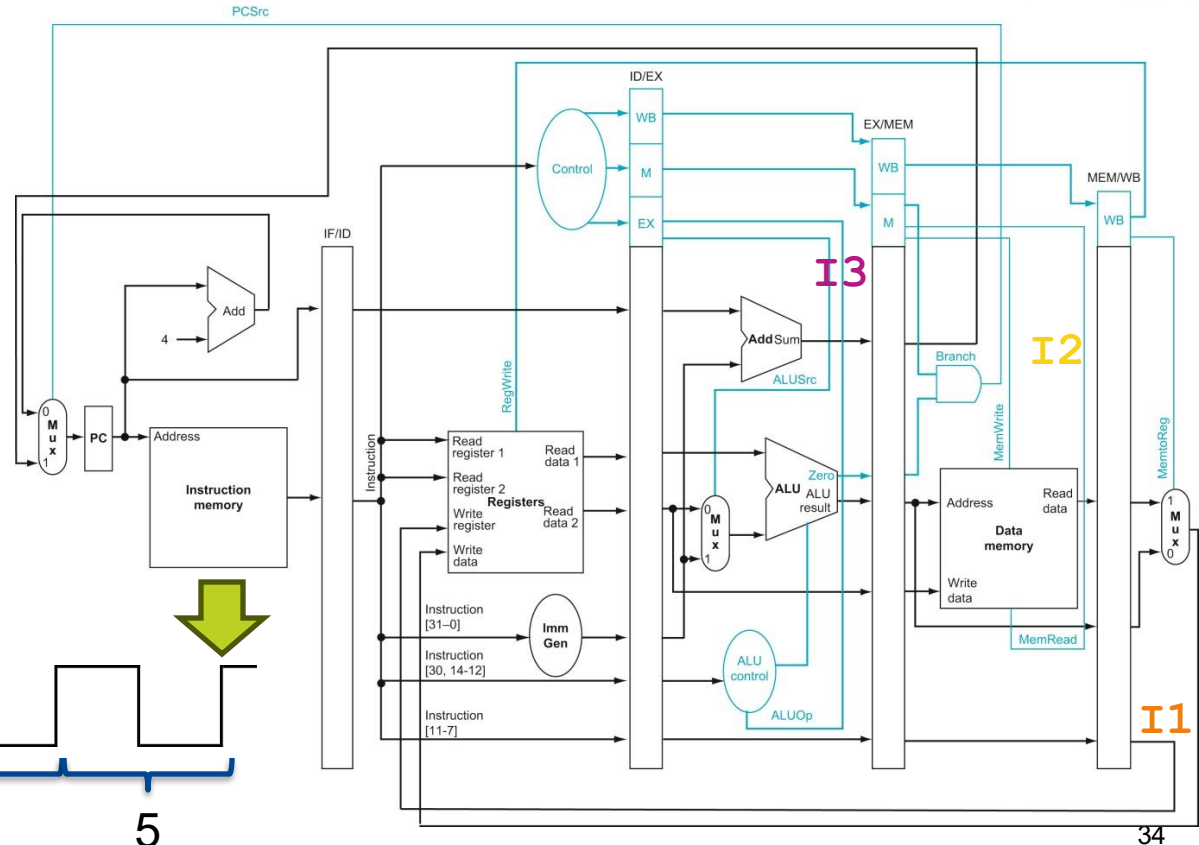
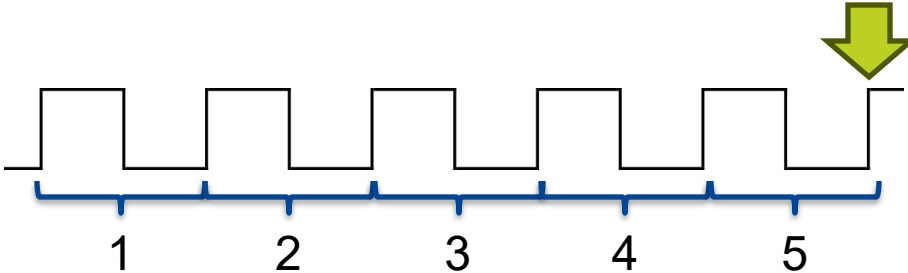
Eksempel: Kontrollsignaler

x1	5	x5	
x2	10	x6	-
x3	15	x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8



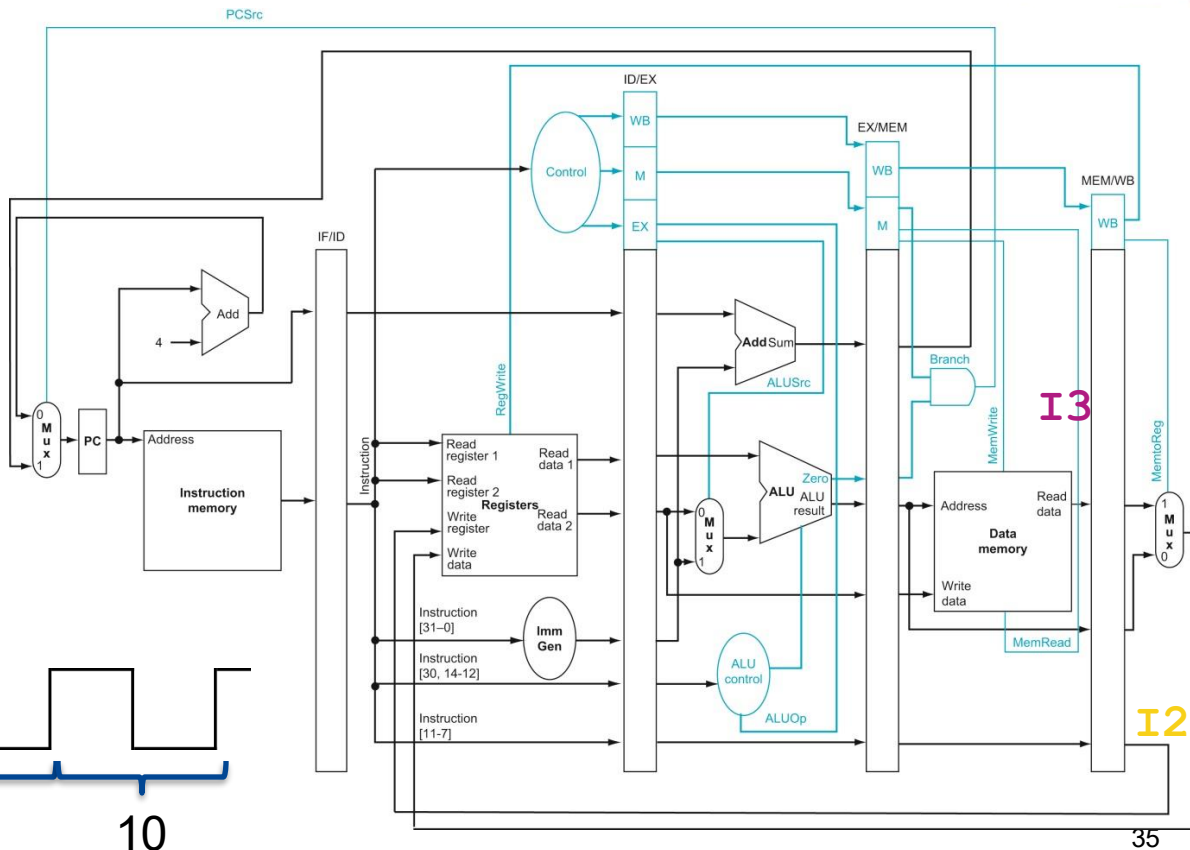
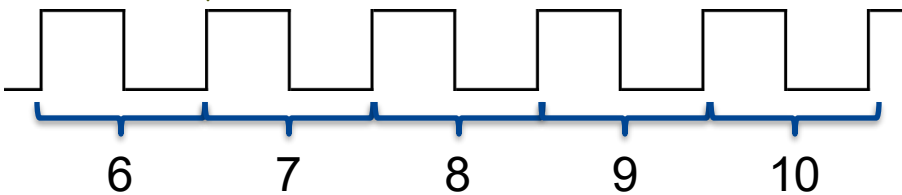
Eksempel: Kontrollsignaler

x1	5	x5	42
x2	10	x6	-
x3	15	x7	25
x4	1024	x8	27

I1 (0) : add x3, x1, x2

I2 (4) : lw x5, 16(x4)

I3 (8) : beq x7, x8, -8





Tema 5.1: Samlebåndsprosessor med 5 steg (Kapittel 4.8)

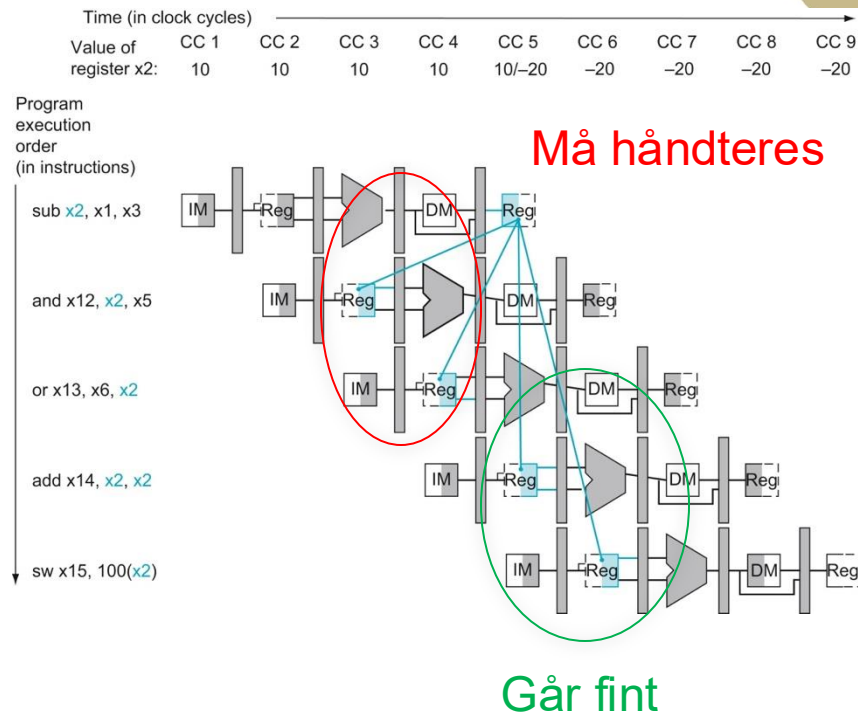
DATAFARER

Læringsutbytte T5.1

- Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessoren sammenliknet med flersykelprosessoren og enkeltsykelprosessoren.
- Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse.
- Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer).
- Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon)
- Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-stegs samlebåndsprosessoren (kunne gjengi og forklare figur 4.43).
- Studenten skal kunne forklare hvordan kontroll implementeres i 5-stegs samlebåndsprosessoren.
- **Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.**
- Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse.

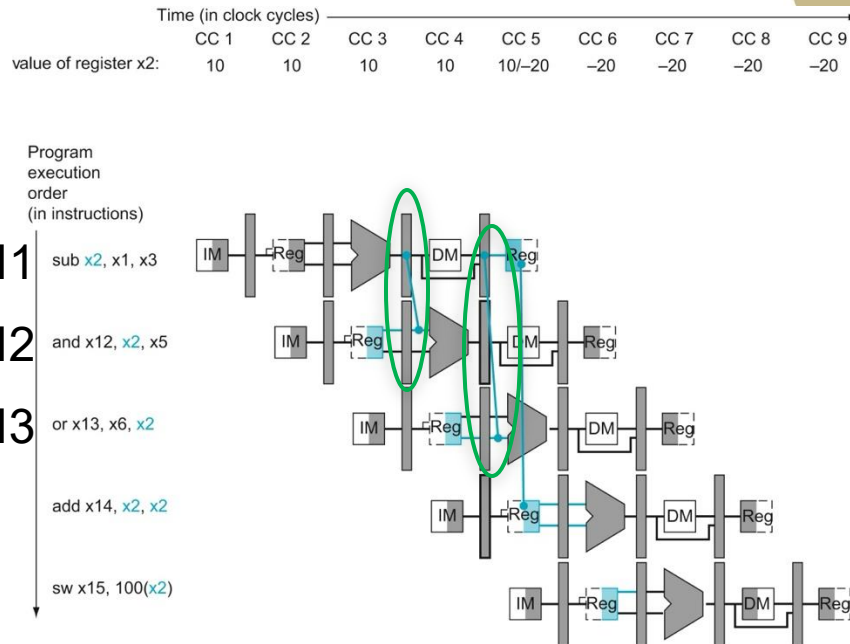
Hvordan håndtere datafarer?

- Vi må håndtere datafarer på en måte som:
 - Oppnår korrekt utføring av alle program
 - Minimerer tap av ytelse
- Datafarer oppnår når:
 - (i) En instruksjon trenger data produsert av en tidligere instruksjon (**avhengighet**)
 - (ii) Instruksjonene utføres nært nok i samlebåndet til at verdiene ikke kan hentes fra registerfila
- Å skrive til registrene i første halvdel av klokkesyklusen og lese i andre halvdel unngår en fare
 - Verdien produsert av `sub x2, x1, x3` er tilgjengelig i registerfila når `add x14, x2, x2` leser den



Videresending («Forwarding»)

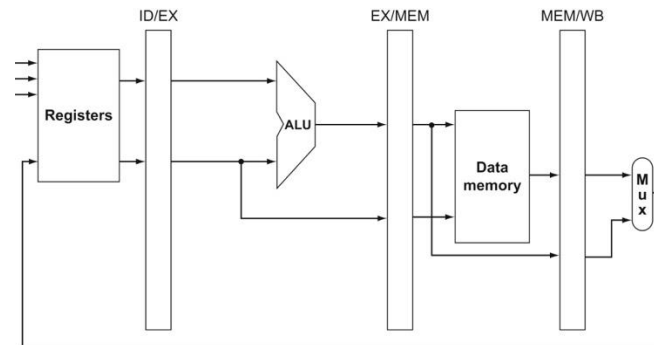
- Observasjon: Verdien en instruksjon trenger er ofte tilgjengelig i et samlebånds-register lenge før den skrives til registerfila
- Løsning: Videresending
 - Vi legger til maskinvare som bytter ut verdien lest fra registerfila med den oppdaterte verdien
 - I2: Rett verdi ligger i EX/MEM
 - I3: Rett verdi ligger i MEM/WB



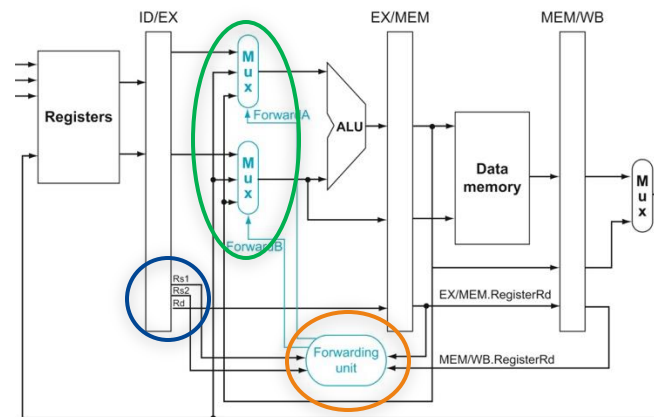
Merk at verdiene videresendes fra venstre mot høyre → **altså fremover i tid**

Implementere videresending

- Vi må lagre registernumrene i samlebåndsregistrene
 - ... fordi nå «trenger vi dem senere»
- Vi må legge logikk som velger den riktige verdien
 - Videresendingsenhet («Forwarding unit»)
- Vi utvider multiplekserene på ALU-inngangene slik at de kan ta imot verdier fra EX/MEM og MEM/WB



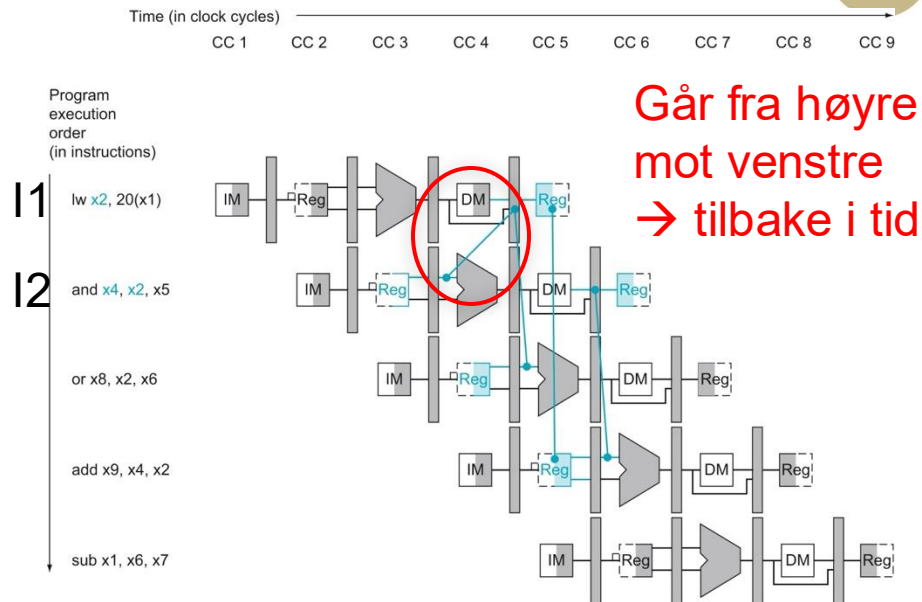
a. No forwarding



b. With forwarding

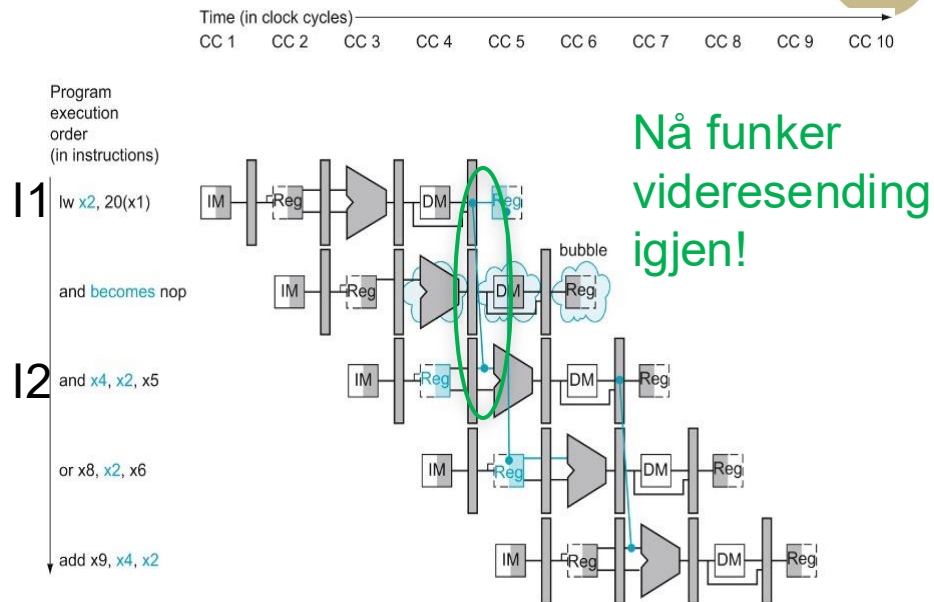
Les-bruk farer («Load-use hazards»)

- Videresending fungerer bare når verdien har blitt produsert når vi trenger den
- Dette eksemplet: I2 trenger verdien i x2 før I1 har hentet den fra minnet
 - Dette er en les-bruk fare
 - Videresending kan ikke brukes fordi vi kan ikke sende verdier tilbake i tid
- Løsning: Vi må holde igjen I2
 - ... med enda mer maskinvare



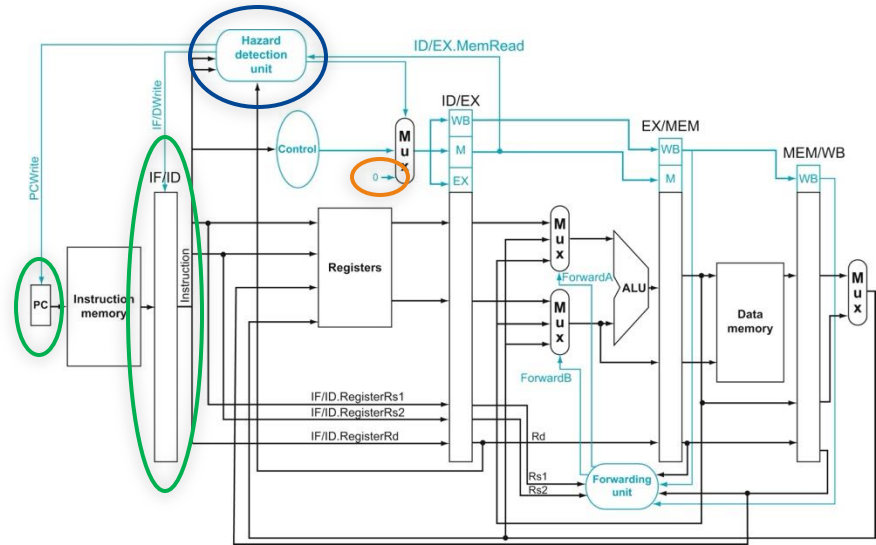
Les-bruk farer («Load-use hazards»)

- Videresending fungerer bare når verdien har blitt produsert når vi trenger den
- Dette eksemplet: I2 trenger verdien i x2 før I1 har hentet den fra minnet
 - Dette er en les-bruk fare
 - Videresending kan ikke brukes fordi vi kan ikke sende verdier tilbake i tid
- Løsning: Vi må holde igjen I2
 - ... med enda mer maskinvare



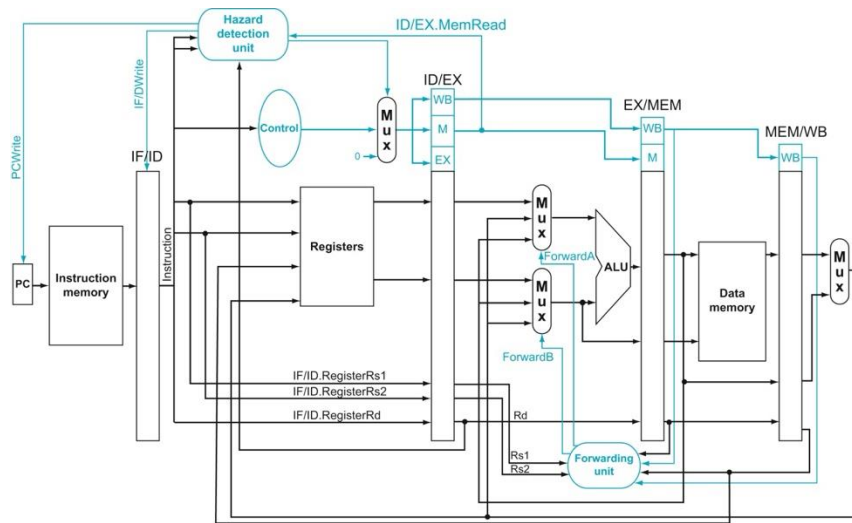
Faredeteksjon («Hazard detection»)

- Faredeteksjonsenheten:
 - Skal vi lese fra minnet?
 - Skal vi skrive til et register som neste instruksjon leser?
- Hvis svaret på begge disse spørsmålene er ja:
 - Lag et hull i samlebandet ved sette (nødvendige) kontrollsignaler til 0
 - Hold samme verdi i PC og IF/ID



Datafarer: Oppsummering

- Vi kan nå håndtere alle datafarer
 - Vi bruker videresending så mye som mulig for å unngå ytelsestap
 - Vi stanser samlebandet når vi må
- Kompilatoren trenger derfor ikke å bry seg om datafarer for korrekthet
 - ... men for ytelse lønner det seg å ikke generere kode med les-bruk farer

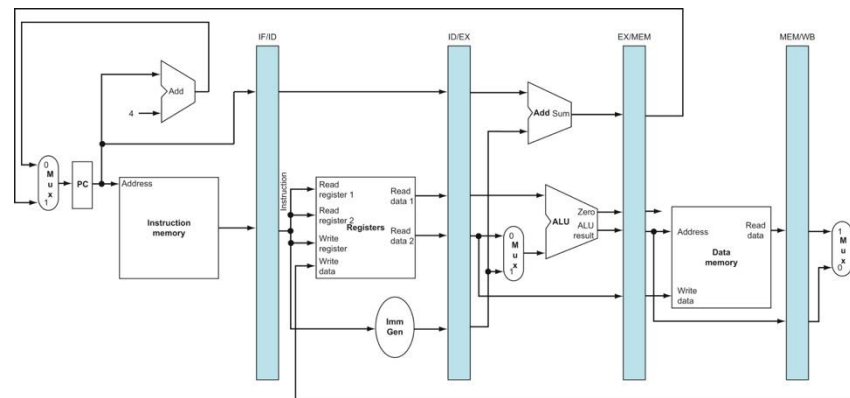




OPPSUMMERING

Oppsummering

- Samlebåndprosessoren er den desidert vanligste arkitekturen vi ser på i TDT4160
 - Gir mye **bedre ytelse** enn enkeltsykel- og flersykelprosessorer med relativt lite ekstra maskinvare
 - Konseptet kan **skaleres** til å håndtere hundrevis av instruksjoner samtidig – og høytytelsesprosessorer er derfor alltid samlebåndarkitekturer
- Samlebåndprosessorer gir noen nye utfordringer
 - Vi har dekket **strukturfarer** og **datafarer** i dag og tar **kontrollfarer** neste uke
 - Vi må også støtte **presise avbrudd**



Figur 4.43